Code Reference

Deep Learning For Beginners: ML Strategies
A Beginner's Guide to Structuring Machine Learning Projects

Book 3 of the AI Foundations Series

WHAT THIS DOCUMENT IS

A complete cross-reference of every code program and snippet used in *ML Strategies*, mapped to the chapter and section where it appears. Each entry lists the file name, the strategy concept it teaches, the libraries it uses, and where to find it in the book and in Appendix B (full listings).

All programs use only scikit-learn and NumPy and run in seconds on an ordinary computer.

1. The Four Core Programs at a Glance

These four programs are the spine of the book. Each is introduced in the chapter shown and listed in full in Appendix B.

Program file	Strategy concept it teaches	Introduced in	Full listing
01_cat_classifier.py	Single-number evaluation metric (precision, recall, F1)	Ch. 3 · §3.4–3.5	Appendix B.1
02_radiology_diagnosis.py	Human-level performance as a Bayes proxy; avoidable bias vs. variance	Ch. 6 §6.4 & Ch. 7 §7.4	Appendix B.2
03_self_driving_detection.py	Multi-task (multi-label) learning — four labels per image	Ch. 14 §14.6	Appendix B.3
04_speech_recognition.py	End-to-end learning vs. a hand-built pipeline; transfer learning	Ch. 15 §15.6	Appendix B.4

SHARED CONVENTIONS ACROSS ALL PROGRAMS

Every program reuses the Book 1 conventions so the numbers stay familiar: `test_size=0.20`, `random_state=42`, `stratify=y` where applicable, and `StandardScaler` fitted on the training set only (no data leakage). Program 2 reuses the Breast Cancer Wisconsin dataset and reproduces Book 1's 98.2% benchmark exactly.

2. Program-by-Program Detail

2.1 01_cat_classifier.py

Introduced	Chapter 3 — Single-Number Evaluation Metrics (§3.4 Program 1, §3.5 Reading the Output)
Full listing	Appendix B.1 (with exercise)
Libraries	numpy, sklearn (make_classification, LogisticRegression, RandomForestClassifier, precision/recall/f1)
Teaches	Why one combined metric (F1) beats juggling precision and recall separately

Key snippet — the single-number reporting helper (Chapter 3):

```
from sklearn.metrics import precision_score, recall_score, f1_score

def report(name, model):
    pred = model.predict(X_dev)
    p = precision_score(y_dev, pred)   # of those CALLED cats, how many were?
    r = recall_score(y_dev, pred)      # of all real cats, how many found?
    f = f1_score(y_dev, pred)          # one number balancing p and r
    print(f"  {name}:  precision={p:.3f}  recall={r:.3f}  -> F1={f:.3f}")
    return f
```

Reported result: Classifier A → F1 0.713; Classifier B → F1 0.878. Appendix B.1 exercise: tune the decision threshold and watch precision, recall, and F1 move.

2.2 02_radiology_diagnosis.py

Introduced	Chapter 6 — Human-Level Performance and Bayes Error (§6.4); revisited in Chapter 7 — Avoidable Bias vs Variance (§7.4)
Also cited	Chapter 9 (reading the first system)
Full listing	Appendix B.2 (with exercise)
Libraries	numpy, sklearn (load_breast_cancer, StandardScaler, LogisticRegression)
Teaches	Splitting error into avoidable bias and variance using a human-level (Bayes) proxy

Key snippet — the bias-vs-variance decision (Chapters 6–7):

```
train_err = 1.0 - model.score(X_train_scaled, y_train)
dev_err   = 1.0 - model.score(X_test_scaled,  y_test)

human_level = 0.005          # 0.5% -- team of experienced doctors (Bayes proxy)

avoidable_bias = train_err - human_level   # gap to the best possible
variance       = dev_err   - train_err     # gap from train to dev

if avoidable_bias > variance:
    print('-> Avoidable bias dominates: focus on FITTING better')
else:
    print('-> Variance dominates: focus on GENERALIZING better')
```

Dev error reproduces Book 1's 98.2% accuracy exactly. Appendix B.2 exercise: raise `human_level` to 0.012 and predict which gap wins before re-running.

2.3 03_self_driving_detection.py

Introduced	Chapter 14 — Multi-Task Learning (§14.6 Make It Concrete)
Full listing	Appendix B.3 (with exercise)
Libraries	numpy, sklearn (make_multilabel_classification, MultiOutputClassifier, LogisticRegression, accuracy_score)
Teaches	One model predicting four labels per image at once; tasks helping each other

Key snippet — four labels per example (Chapter 14):

```
from sklearn.multioutput import MultiOutputClassifier

LABELS = ["pedestrian", "car", "stop_sign", "traffic_light"]

# Each "image" (feature row) carries 4 independent yes/no labels at once.
X, Y = make_multilabel_classification(n_samples=1500, n_features=20,
                                      n_classes=4, n_labels=2, random_state=42)
X_tr, X_dev, Y_tr, Y_dev = train_test_split(X, Y, test_size=0.20,
                                            random_state=42)
```

Reported per-task dev accuracy: pedestrian 85.3%, car 83.7%, stop_sign 82.0%, traffic_light 78.3%. Appendix B.3 exercise: remove one label and re-check the remaining accuracies.

2.4 04_speech_recognition.py

Introduced	Chapter 15 — End-to-End Deep Learning (§15.6 Make It Concrete)
Full listing	Appendix B.4 (with exercise)
Libraries	numpy, sklearn (load_digits, PCA, make_pipeline, LogisticRegression)
Teaches	End-to-end learning vs. a hand-built pipeline, plus transfer learning

Key snippet — pipeline vs. raw input setup (Chapter 15):

```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

# Digits (8x8 = 64 "signal" features) stand in for short audio clips -> labels.
X, y = load_digits(return_X_y=True)
X_tr, X_dev, y_tr, y_dev = train_test_split(X, y, test_size=0.20,
                                            random_state=42)
```

Reported dev results: pipeline (PCA → classifier) 93.3%; end-to-end (raw → classifier) 95.8%; transfer learning on a data-poor task (digits 8–9) 97.2%. Appendix B.4 exercise: shrink the training set and watch the pipeline-vs-end-to-end gap change.

3. Supporting Program

3.1 data_mismatch_demo.py

Introduced	Chapter 12 — Addressing Data Mismatch (§12.7 Make It Concrete)
Libraries	numpy, sklearn (load_breast_cancer, StandardScaler, LogisticRegression)
Teaches	Using a training-dev set to tell data mismatch apart from a plain variance problem

The program reuses the Book 1 dataset and conventions, then deliberately adds noise to the dev/test data so train and dev come from different distributions. Carving out a training-dev set (same distribution as training, but not trained on) separates the three gaps cleanly:

```
Human-level (best transcribers):   0.5%
Training error:                    1.0%
Training-dev error:                5.1%
Dev error:                         5.6%

Avoidable bias  = 1.0% - 0.5%  =  0.5%   (human    -> training)
Variance        = 5.1% - 1.0%  =  4.1%   (training -> train-dev)
Data mismatch   = 5.6% - 5.1%  =  0.5%   (train-dev -> dev)
```

4. Inline Code Snippets (not part of the four programs)

Short snippets embedded in the running text to illustrate a single idea.

Snippet	What it shows	Location
Stratified train/dev split	`train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)`	Ch. 5 §5.4
Bias/variance decision logic	Computing avoidable bias and variance from train/dev error	Ch. 7 §7.3

5. Quick Chapter → Code Cross-Walk

Chapter	Code referenced
Ch. 3	01_cat_classifier.py
Ch. 5	Stratified split snippet (inline)
Ch. 6	02_radiology_diagnosis.py (introduced); ml_demo.py referenced (Book 1)
Ch. 7	02_radiology_diagnosis.py (bias-vs-variance decision); bias/variance snippet
Ch. 9	02_radiology_diagnosis.py (reading the first system)
Ch. 12	data_mismatch_demo.py

Chapter	Code referenced
Ch. 14	03_self_driving_detection.py
Ch. 15	04_speech_recognition.py
Appendix B	Full listings + exercises for all four core programs